Master Cheat Sheet — Python Dev Environment

30 Commands | 7 Topics

venv pip conda poetry uv .env docker

TOPIC	COMMAND	DESCRIPTION
venv	`python -m venv myenv`	Create virtual environment
	`source myenv/bin/activate`	Activate (Linux / Mac)
	`myenv\Scripts\activate`	Activate (Windows)
	`deactivate`	Exit the environment
pip	`pip install package`	Install a package
	`pip install package==1.2.3`	Install specific version
	`pip install -r requirements.txt`	Install all dependencies
	`pip freeze > requirements.txt`	Save current deps to file
	`pip list`	Show installed packages
	`pip show package`	Show package details
conda	`conda create -n env python=3.11`	Create conda environment
	`conda activate env`	Activate conda env
	`conda deactivate`	Deactivate conda env
	`conda env list`	List all environments
	`conda env create -f environment.yml`	Create from YAML file
	`conda install package`	Install via conda
poetry	`poetry new myproject`	Create new project
	`poetry add package`	Add a dependency
	`poetry add --group dev pytest`	Add dev dependency
	`poetry install`	Install from lock file
	`poetry run python app.py`	Run inside poetry env
uv	`uv venv`	Create env with uv
	`uv pip install package`	Fast package install
	`uv pip install -r requirements.txt`	Fast bulk install
.env	`load_dotenv()`	Load .env in Python
	`os.environ.get('KEY')`	Read env variable
	`os.environ.get('KEY', 'default')`	Read with default value
docker	`docker build -t app .`	Build Docker image
	`docker run app`	Run container
	`docker run --env-file .env app`	Run with env file

Recommended Workflow — Every New Project

8 Steps | ~5 min

1 Create Project Folder

`mkdir myproject`

Every project gets its own dedicated directory. Never mix projects.

```
mkdir myproject
cd myproject
```

2 Create Virtual Environment

`python -m venv venv`

Isolate this project's dependencies from system Python & other projects.

```
python -m venv venv
```

3 Activate the Environment

`source venv/bin/activate`

Prompt shows (venv). All pip installs go here only — not globally.

```
source venv/bin/activate    # Linux / Mac
venv\Scripts\activate       # Windows
```

4 Install Your Packages

`pip install ...`

Install only what this project needs. Keep it minimal and clean.

```
pip install flask requests python-dotenv sqlalchemy
```

5 Save Dependencies

`pip freeze > requirements.txt`

Lock exact versions — teammates get the identical environment.

```
pip freeze > requirements.txt
```

6 Set Up .env File

`touch .env`

Store secrets in .env. Provide .env.example with dummy values for team.

```
# .env  (never commit!)
SECRET_KEY=abc123
DATABASE_URL=postgresql://localhost/db
```

7 Organise Project Structure

`mkdir src tests`

Keep source in a package. Separate models, routes, utils, and tests.

```
src/__init__.py  models/  routes/  utils/
tests/   README.md   .gitignore
```

8 Commit to Git

`git init && git add .`

Track everything except .env. Your repo is now reproducible anywhere.

```
git init
git add .
git commit -m 'Initial project setup'
```

Activate venv before every pip install

Never commit .env — use .env.example

Pin versions in requirements.txt

One virtual env per project always